



兰州大学

本科毕业论文

论文题目 (中文) 基于 Julia 的单细胞
数据分析平台的设计与实现

论文题目 (英文) Design and Implementation of a Julia-based
Single-Cell Data Analysis Platform

学生姓名 李泽华
指导教师 赵伟
学 院 生命科学学院
专 业 生物信息学
年 级 2021 级

兰州大学教务处

诚信责任书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所取得的成果。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人、集体已经发表或未发表的论文。

本声明的法律责任由本人承担。

论文作者签名： 李泽华 日 期： 2025 年 4 月 27 日

关于毕业论文（设计）使用授权的声明

本人在导师指导下所完成的论文及相关的职务作品，知识产权归属兰州大学。本人完全了解兰州大学有关保存、使用毕业论文（设计）的规定，同意学校保存或向国家有关部门或机构送交论文的纸质版和电子版，允许论文被查阅和借阅；本人授权兰州大学可以将本毕业论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业论文（设计）。本人离校后发表、使用毕业论文（设计）或与该毕业论文（设计）直接相关的学术论文或成果时，第一署名单位仍然为兰州大学。

本毕业论文（设计）研究内容：

☒ 可以公开

☐ 不宜公开，已在学位办公室办理保密申请，解密后适用本授权书。

（请在以上选项内选择其中一项打“√”）

论文作者签名： 李泽华 导师签名： 赵伟

日 期： 2025 年 4 月 27 日

日 期： 2025 年 4 月 30 日

基于 Julia 的单细胞数据分析平台的设计与实现

中文摘要

单细胞转录组测序 (scRNA-seq) 等单细胞技术正以前所未有的速度飞跃发展, 推动生命科学进入以细胞为基本单元深入解析的新时代。相较于传统的群体测序 (bulk RNA-seq), 单细胞测序能够捕捉个体细胞之间的异质性, 揭示更为精细的生物学图景。然而, 由于数据的高维性、稀疏性及噪声等特性, 其分析对计算工具与算法提出了更高要求。

目前, Scanpy (Python)、Seurat (R) 等主流分析平台虽功能完备, 但在性能方面受限于解释性语言的运行效率。为追求高性能, 它们往往依赖 C/C++ 或 Rust 等静态类型语言进行底层加速, 这不仅加重了使用负担, 也提高了调试的门槛。

本研究提出 Juscan.jl 是一个完全使用 Julia 编写的单细胞数据分析平台, 旨在打破性能瓶颈与开发复杂度之间的权衡。基于 Muon.jl 提供的 AnnData 数据结构, Juscan.jl 实现了包括数据预处理、高变基因识别、降维与聚类在内的一站式分析流程。平台设计力求模块化与简洁性, API 风格对标 Scanpy, 降低用户迁移成本, 并可充分利用 Julia 的 JIT 编译与原生并行机制, 实现高性能计算。

实验结果表明, 在大规模数据集处理上, Juscan.jl 的效率明显优于 R 语言的 Seurat, 虽初版性能略低于 Scanpy, 但 Julia 语言本身具备更高的性能上限, 随着后续优化有望显著提升。本研究不仅为 Julia 在单细胞领域的应用奠定基础, 也为高性能生物数据分析提供了新思路与工具支持。

关键词: 单细胞转录组测序 (scRNA-seq), 生物信息学, Julia, 高性能计算, 数据分析工具

Design and Implementation of a Julia-based Single-Cell Data Analysis Platform

Abstract

Single-cell technologies, such as single-cell RNA sequencing (scRNA-seq), are advancing at an unprecedented pace, ushering life sciences into a new era of analysis at cellular resolution. Compared to traditional bulk RNA sequencing, single-cell approaches enable the capture of cellular heterogeneity, revealing a much more refined landscape of biological processes. However, due to the high dimensionality, sparsity, and inherent noise of such data, their analysis places stringent demands on computational tools and algorithmic frameworks.

Currently, mainstream platforms like Scanpy (Python) and Seurat (R) offer comprehensive functionalities but are limited in performance due to the runtime inefficiencies of interpreted languages. To overcome these limitations, many of these tools rely on static languages such as C/C++ or Rust for backend acceleration—an approach that increases development complexity and raises the barrier for debugging and maintenance.

In this study, we introduce Juscan.jl, a single-cell data analysis platform written entirely in Julia, aiming to break the trade-off between computational performance and development simplicity. Built upon the AnnData structure provided by Muon.jl, Juscan.jl delivers an end-to-end pipeline for single-cell analysis, encompassing data preprocessing, highly variable gene selection, dimensionality reduction, and clustering. The platform adopts a modular and minimalist design, with an API style inspired by Scanpy to ease user migration, while fully leveraging Julia’s just-in-time (JIT) compilation and native parallelism for high-performance computing.

Experimental results show that Juscan.jl significantly outperforms Seurat when processing large-scale datasets. While its current version slightly trails Scanpy in performance, Julia’s inherently higher performance ceiling offers promising potential for future optimization. This work not only lays a foundation for Julia’s application in single-cell transcriptomics but also offers new perspectives and tools for high-performance biological data analysis.

Keywords: Single-cell RNA sequencing (scRNA-seq), bioinformatics, Julia, high-performance computing, data analysis tools

目 录

中文摘要	I
英文摘要	II
第一章 引言	1
1.1 研究背景	1
1.1.1 单细胞数据分析及其在生物学的重要性	1
1.1.2 单细胞数据分析概述	1
1.1.3 现有单细胞分析平台	5
1.2 研究意义与目标	6
1.2.1 研究意义	6
1.2.2 本项目的目标	7
第二章 系统设计与实现	9
2.1 开发环境与工具链	9
2.1.1 编程语言	9
2.1.2 依赖库	9
2.1.3 开发工具	9
2.2 Juscan.jl 总体架构设计	9
2.2.1 整体流程图	9
2.2.2 系统模块划分	10
2.3 数据结构	11
2.4 数据预处理模块	12
2.4.1 计算质量控制指标	12
2.4.2 过滤	13
2.4.3 归一化	14
2.5 工具模块	15

2.5.1	高可变基因	15
2.5.2	降维	16
2.5.3	聚类	17
2.6	可视化模块	19
2.6.1	调色机制与主题色板	19
2.6.2	可视化函数接口	19
第三章	实验与评估	22
3.1	实验数据集介绍	22
3.2	实验设计与流程	22
3.2.1	数据读取与质量控制	22
3.2.2	数据归一化与高变基因	24
3.2.3	降维与聚类	25
3.3	性能评估	26
3.3.1	Seurat 对比 juscan.jl 评估结果	27
3.3.2	scanpy 对比 juscan.jl 评估结果	27
第四章	讨论	28
4.1	研究成果总结	28
4.2	未来工作展望	28
参考文献		29
附 录		30
A.1	Juscan.jl 源代码与文档	30
A.2	性能检测	30
致 谢		45

图 目 录

图 2.1	Juscan.jl 流程图	10
图 2.2	anndata 数据结构示意图	12
图 3.1	Juscan.jl 质量控制可视化结果.....	24
图 3.2	Juscan.jl 归一化和高变基因可视化结果	25
图 3.3	Juscan.jl 聚类结果 umap 可视化	26
图 3.4	Juscan.jl 与 Seurat 性能对比.....	27
图 3.5	Juscan.jl 与 Scanpy 性能对比	27

表 目 录

表 1 单细胞 RNA-seq 数据中先进聚类方法概述 1	3
表 2 不同聚类方法的特性比较.....	4
代码 1: 文件结构.....	10
代码 2: Muon.AnnData	11
代码 3: Muon.Pp.calculate_qc_metrix	12
代码 4: Muon.Pp.filter.....	13
代码 5: Muon.Pp.normalize.....	14
代码 6: Muon.Tl.highly_variable_genes.....	16
代码 7: Muon.Tl.pca.....	17
代码 8: Muon.Tl.umap	17
代码 9: Juscan.Tl.Clustering.....	18
代码 10: Muon.Pl.violin	20
代码 11: Muon.Pl.plot_umap	20
代码 1: Juscan.jl 示例: 数据读取与质量控制	22
代码 2: Juscan.jl 示例: 数据归一化与高变基因	24
代码 3: Juscan.jl 示例: 数据降维与聚类.....	25

第一章 引言

1.1 研究背景

1.1.1 单细胞数据分析及其在生物学的重要性

著名细胞学家 E. B. Wilson 曾在他的著作 *The Cell in Development and Heredity* 中强调 "The key to every biological problem must finally be sought in the cell."，即便是生物化学，分子生物学，遗传学高度发达的今天，我们不但在研究几乎任何生命现象的过程中都离不开细胞的基本单位，而且从细胞的角度进行研究比以往任何时间都有现实意义。

在 Wilson 提出该观点的年代，生物学的研究主要依靠显微镜和简单的生化实验来探讨细胞和亚细胞单位的异质性和功能，这一过程主要依赖形态学进行分析，难以揭示细胞的分子差异。而后出现的群体测序 (bulk RNA-seq)，也只能检测细胞群体的平均状态，无法区分单个细胞的差异性。

随着分子生物学的发展，人类逐渐了解了 DNA，RNA 的结构，并可以通过桑格测序，二代测序方法获取核酸序列信息。现代的单细胞分析技术，使用优化的二代测序技术，以较高的单细胞分辨率，检查单个细胞的核酸序列。以转录组测序 (scRNA-seq) 为例，首先使用荧光激活细胞分选术 (FACS) 或微流体设备^[1] 高通量的快速分离数百至数十万个细胞，然后通过群体测序 (bulk RNA-seq) 类似的步骤进行测序，即逆转录 (RT)、扩增、文库生成和测序^[2]，这里不做展开。

可以看到，单细胞技术使得 Wilson 的观点进入了可操作的时代。单细胞转录组测序 (scRNA-seq)、单细胞表观遗传测序 (ATAC-seq)、空间转录组学 (spatial transcriptomics) 等方法，使得现代的研究可以真正在单个细胞的分辨率下探索生物学问题。

对于单细胞测序获得的大规模数据，需要通过生物信息学的分析方法研究。单细胞数据分析包括从数据预处理、降维、聚类、伪时序分析到多组学整合的多个环节，现有许多计算工具来支持这一研究领域。其中，Scanpy^[3]、Seurat、AnnData 以及 SingleCellExperiment 等成为主流，推动了单细胞数据的标准化并降低了数据分析门槛。

1.1.2 单细胞数据分析概述

scRNA-seq 是一套流行而强大的单细胞检测和分析技术，可以让研究者以单个细胞为单位获取并分析大量细胞整个转录组。但这同时也带来了巨大的数据集，需要使用专业的统计和分析流程。scRNA-seq 分析围绕表达矩阵展开，即 `Anndata.X`，代表了每个基因和细胞中观察到的转录本数量。以表达矩阵为中心进行数学建模设计和搭建一整套统计和分析算法。

(1) 数据预处理

在开始真正的数据分析之前，为了保证下游分析结果的准确性，需要对数据进行基础的预处理。庞大的表达矩阵中，由于技术精度等问题，优质的数据与噪音，杂质等夹杂在一起。可以通过设定一个数据集特定的 UMI 数量阈值来过滤。确保只保留那些显著偏离背景水平的条形码，从而捕获那些 RNA 含量较低但真实存在的细胞。

但这种初步的筛选无法排除受损和死亡的细胞，此时就需要通过计算一些质量控制指标来识别，这些指标通常包括每个细胞中表达的基因数，每个基因在多少个细胞中被表达，线粒体来源 RNA 的比例以及无法比对或多重比对的 reads 比例等。

此外，对于 scRNA-seq 来说，无论是高通量低深度的 10X 方法还是低通量高深度的 Smart-seq2 协议，因为每个细胞的 RNA 量会因细胞周期阶段和其他生物因素而有很大差异，导致从测序实验中获得的有用读数的数量差异显著。为了矫正这种差异，会对整体的 scRNA-seq 数据计算一个与样本测序升读相关的量，即 *size factor*，然后令整个表达矩阵除以该值。

(2) 高变异基因

识别高变异基因 (highly variable genes/HVG) 是为了通过过滤掉信息量较少的特征来解决高维数据中固有的维数灾难，方便包括降维，聚类，伪时序分析等在内的下游分析，使算法能够专注于最具信息量的特征来描述细胞的异质结构。也因此成为了决定分析效果的重要步骤。

Cell Ranger 方法通过对数归一化后的表达值计算离散度，高离散度的基因被选中作为 HVGs，展现出它们在细胞间独特的变异性。而根据 *seurat* 的经验，*seurat_v1* 和 *seurat_v2* 使用归一化表达矩阵在计算离散度，通过不同的离散度计算方法，筛选出表达矩阵中高变异的基因。而 *seurat_v3* 使用原始计数构建出一个对数方差和对数均值之间的拟合曲线，通过曲线精准的识别偏离常规编译模式的基因。

(3) 降维

减少表达矩阵高维性负面影响的另一种策略是在减少的特征空间上执行降维。其中最常用的方法是 PCA，这是一中线性降维方法，能够保留细胞间的欧几里得距离，并在大规模数据集中迅速计算出重要的成分，也就是方差最大的方向。需要保留的维度数是 PCA 的一个重要参数，取决于数据的复杂度，常见的方法是根据“肘部法则”，绘制每个成分解释的方差比例图，选择曲线上肘部的点筛选出最具代表性的成分。

然而即便如此，单独的几个主成分仍然难以代表 scRNA-seq 数据的全部表达矩阵，特别是对于复杂、弯曲的流形结 (manifold structure)，因此，需要依靠可视化算法将多维信息映射到二维平面上。当前最流行的就是 Uniform Manifold Approximation and Projection (UMAP)，它构建细胞间的近邻网络，来近似模拟数据的拓扑结构，再使用高斯核函数计

算邻接概率，如公式1所示，并通过最小化一个交叉熵（Cross Entropy）形式的损失函数即公式2，在低维空间中模拟重现高维结构。

$$p_{ij} = \exp\left(-\frac{||x_i - x_j||^2}{\sigma_i}\right)$$

(1)

$$L = \sum_{i \neq j} \left[p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) + (1 - p_{ij}) \log\left(\frac{1 - p_{ij}}{1 - q_{ij}}\right) \right]$$

(2)

其中 p_{ij} 表示低维空间中点 i 和点 j 由 sigmoid-like 函数计算得来的相似度。

另一个常见的方法是 t-distributed stochastic neighbor embedding（t-SNE），但他在保留大规模数据上的能力不足 UMAP。同时需要注意的是，不论是 UMAP 还是 t-SNE 都需要用户提供的超参数，并且运算结果对该参数非常敏感。

(4) 聚类

细胞聚类是一个把细胞集合划分成子集的过程，每个子集是一个簇，使得簇中的对象彼此相似，但与其他簇中对象彼此相异。

数据聚类等无监督学习随着单细胞测序技术的进步，已经成为识别细胞类型和发现新细胞的主要方法。当前生物信息学针对单细胞 RNA 序列数据已经开发了多种类型的聚类方法，主要分为 k 均值聚类，层次聚类，社区发现以及基于密度的聚类^[4]。下面的两个表1，??总结了当前常用的聚类方法的优缺点以及特点。

表 1 单细胞 RNA-seq 数据中先进聚类方法概述 1

方法	类型	优点	缺点
SAIC	k-means	复杂度低；可扩展至大数据规模	对离群值敏感；无法估计聚类数量
RaceID	k-means	对稀有细胞类型敏感；可估计聚类数量	不适用于稀有细胞类型
pcaReduce	k-means/层次聚类	可提供层次结构的聚类结果	不稳定；不适用于大规模数据
SC3	k-means/层次聚类	准确性高；可估计聚类数量	复杂度高
CIDR	层次聚类	对 dropout（数据缺失）敏感	复杂度高
BackSPIN	层次聚类	可同时对基因和细胞进行聚类	复杂度高

表格接下页

续表: 单细胞 RNA-seq 数据中先进聚类方法概述 1

方法	类型	优点	缺点
SIMLR	谱聚类 (Spectral)	适用于异质性和噪声数据	不适用于大规模数据
SinNLRR	谱聚类 (Spectral)	适用于噪声数据	无法估计聚类数量
SCANPY	Louvain	复杂度低; 可扩展至大规模数据	可能无法识别小的群体
Seurat	Louvain	复杂度低; 可扩展至大规模数据	可能无法识别小的群体
GiniClust	基于密度的方法	可检测稀有细胞类型	对大型聚类不敏感

表 2 不同聚类方法的特性比较

方法	复杂度	可自动确定聚类数	可扩展性	可检测稀有聚类
SAIC	$O(N \cdot K \cdot T)$	✗	✓	✗
RaceID	$O(N \cdot K \cdot T)$	✓	✓	✓
pcaReduce	$O(N \cdot K \cdot T) / O(N^3)$	✓	✗	✗
SC3	$O(N \cdot K \cdot T) / O(N^3)$	✓	✗	✗
CIDR	$O(N^3)$	✓	✓	✓
BackSPIN	$O(N^3)$	✗	✗	✓
SIMLR	$O(N^3)$	✓	✗	✗
SinNLRR	$O(N^3)$	✓	✗	✗
SCANPY	$O(N \log N)$	✓	✓	✗
Seurat	$O(N \log N)$	✓	✓	✗
GiniClust	$O(N \log N)$	✗	✓	✓

注: N 表示样本数量, K 表示聚类簇数, T 表示迭代次数。时间复杂度 O , 描述算法运行所需的时间, 通常取决于输入数据的规模 n 。

a. k 均值聚类

k-means 是流行的聚类方法中最简单的, 需要预定义聚类中心, 首先随机初始化每个单元的质心。通过不断迭代来最小化质心与其单元各点之间的欧氏距离的平方和。此方法可以随着数据点的数量线性拓展, 因此适用于大型数据。但问题在于 k-means 是一种贪婪

算法，容易陷入局部最优解，而且选择较大的 k 值，会提高分群的准确度，但同时会增大过拟合的风险。

b. 社区发现

社区发现算法在社会学和生物学数据这样可以被表示为具有节点和边的图系统中是一种十分流行的聚类算法。对于单细胞 RNA 数据，节点指的是细胞，边权重为细胞之间在表达空间里的亲近度。Louvain 是最流行的社区发现算法，广泛用于单细胞 RNA 序列数据^[5]，常用的 Scanpy，Seurat 包都提供了 Louvain 的聚类方法对单细胞进行聚类。专注于优化模块度（modularity）函数，揭示图中高度连接的子结构——即“社区”。

Louvain 递归地将社区合并为单个节点，即先对原始图进行社区划分，找到局部模块度最优的社区结构，接着，把每一个识别出来的社区看作是图中的一个特殊节点，将整张图重新构造为一个新的压缩图。接下来在压缩图上执行模块度聚类，再次执行模块度优化算法，寻找新的社区划分。Louvain 的时间复杂度为 $O(N \log N)$ ，显著低于其他社区发现算法。

$$H = \frac{1}{2m} \sum_c \left[e_c - \gamma \frac{k_c^2}{2m} \right] \quad (3)$$

如公式3所示，模块度 H 是一个度量图结构中社区划分优劣的指标，越高表示社区划分越合理。

1.1.3 现有单细胞分析平台

近年来，随着单细胞 RNA 测序（scRNA-seq）技术的快速发展，一系列面向单细胞数据分析的计算工具应运而生，并在科研社区中建立起广泛的用户基础。其中，Scanpy (Python)、Seurat (R) 以及 SingleCellExperiment (R) 是目前应用最为广泛的三个分析平台，各自具备鲜明的设计理念与技术优势。

(1) Seurat

Seurat^[6,7] 是 R 语言中最具代表性的单细胞分析工具之一，自发布以来持续更新，目前已发展至第五版。它采用模块化设计，涵盖从质量控制、归一化、特征选择，到聚类、降维与可视化的完整流程。Seurat 的一大特色是其对复杂生物结构的建模能力，尤其是在样本整合、空间转录组、轨迹推断等方向中具有领先优势。此外，Seurat 提出了如“anchor”、“dictionary learning”等创新性算法，为多组学与参考映射提供了解决方案。

然而，Seurat 的运行效率相对受限，尤其在处理大型数据集时存在内存占用高、执行缓慢的问题。尽管开发团队通过引入 C++ 加速模块进行优化，但其主框架依然依赖于解释性语言 R，在高性能计算方面存在一定局限。

(2) scanpy

Scanpy^[8] 是 Python 语言中最主流的单细胞分析工具，构建在 AnnData 数据结构之上，具有良好的可扩展性与模块兼容性。其设计理念强调“轻量级、高性能、面向大数据”，适合处理十万级以上细胞的 scRNA-seq 数据。同时，Scanpy 与 Muon 框架联合提供了对多模态数据（如 RNA+ATAC）的支持^[9]。

相比 Seurat，Scanpy 拥有更快的处理速度和更自由的编程能力，但其函数接口相对低层，用户需具备较强的编程背景。更重要的是，Scanpy 在底层实现中大量调用了 Numba、Cython 和 C++ 等异构技术，尽管提升了性能，但也增加了调试难度和系统依赖复杂性。

(3) SingleCellExperiment

SingleCellExperiment^[10] 并非一个完整分析工具，而是 R/Bioconductor 生态中用于组织单细胞数据的标准数据结构。它以 SummarizedExperiment 为基础，提供统一的容器形式用于存储表达矩阵、元数据和降维结果。该结构被广泛用于支持 Bioconductor 生态中的各类单细胞工具，如 scater、scran、batchelor 等，是构建可复现分析流程的重要基础。

与 Scanpy 和 Seurat 相比，SingleCellExperiment 更注重数据结构的标准化和模块解耦，适合构建灵活、可插拔的分析工作流。然而，它缺乏统一的“主框架”，用户需组合多个包使用。

1.2 研究意义与目标

1.2.1 研究意义

近年来，由于 python 语言的简单易用，以及其丰富的第三方库，使得 python 成为了数据分析领域最热门的语言。因而基于 python 产生了众多单细胞库，例如 anndata^[9]，scanpy^[3]。

但同时，python 在也因为其极低的运行效率饱受诟病。这主要是因为其动态类型，解释运行和难以多核并发等特性所导致的。从传统意义上来说，较高的运行效率是以牺牲开发效率为代价的，例如 C/C++，但是随着编程语言技术的发展，一种新的编译器框架技术即 LLVM(Low-Level Virtual Machine) 的出现，使得计算机可以在不牺牲开发效率的情况下获得较高的运行效率。

Julia^[11, 12] 是一门新兴的数据科学语言，使用 LLVM 作为其核心编译器基础设施，它为 julia 提供了一个高度优化的编译器框架，使 Julia 能够将代码编译成高效的机器码，从而提供接近 C 和 Fortran 的运行性能。Julia 还引入了 JIT(Just-In-Time) 即时编译技术，由于 LLVM 支持即时 (JIT) 编译，Julia 可以在运行时进行动态编译。这意味着，Julia 是一种动态的强类型语言，变量一旦被赋予某种类型，该类型在运算过程中不会改变，编译器可以在编译期间推断出变量和表达式的具体类型，同时 JIT 编译器可以在生成代码时进行高度优化，避免诸如类型检查或类型转换等操作。另外，python 的 GIL(Global Interpreter Lock) 机制使其无法充分利用多核 CPU，而 Julia 可以充分并方便利用多核 CPU，从而提高运行效率，特别是对于单细胞处理中质控，降维，聚类这些 CPU 密集型任务。

但在 Julia 生态系统中，单细胞分析工具尚处于初步发展阶段，社区尚未形成完善的支持体系，第三方库的数量和功能也相对有限。

目前，Julia 平台上已有一些小型的单细胞分析库。例如，Muon.jl 是由 Anndata 官方团队开发的 Julia 版本，旨在提供与 Python 中的 Muon 类似的数据结构。然而，Muon.jl 主要专注于数据结构的实现，缺乏完整的分析工具链，用户需要自行实现数据预处理、降维、聚类等分析步骤。

此外，Julia 社区中还存在一些功能较为单一的库，专注于单细胞分析的特定方面。例如，scVI.jl^[13] 是一个用于拟合变分自编码器（VAE）模型的 Julia 包，基于 Python 的 scvi-tools 实现，主要用于处理计数分布的单细胞数据。该库提供了标准和线性解码的 VAE 模型^[14]，支持负二项分布、泊松分布、高斯分布和伯努利分布等生成分布，以及不同的离散参数设置。

还有一些库，如 ASCT.jl^[15]，虽然实现了类似 Seurat 的功能，提供了从质量控制、预处理、降维、聚类、标记基因识别到样本整合的完整流程，但并未基于官方的 Muon.jl 结构，而是采用了自定义的数据结构。这种做法可能导致与其他工具的兼容性问题，限制了数据的共享和复用。

综上所述，虽然 Julia 在性能和语法方面具备显著优势，但其在单细胞分析领域的工具生态仍不够成熟，缺乏功能全面、结构统一的分析平台。正因为如此，开发一个基于 Julia 的全栈单细胞分析工具，既是对现有生态的补足，也是推动高性能生物信息学工具发展的关键契机。

随着测序技术的飞速进步，单细胞数据的规模持续膨胀，传统工具在分析效率上的瓶颈逐渐显现。高性能工具的缺位带来了计算资源浪费与能源开销的增加，成为科研中的隐性成本。而 Julia 所带来的运行效率提升，不仅可以缩短实验周期，加快结果反馈，有助于快速迭代实验与建模，也意味着更低的资源消耗，节省服务器运行成本，降低科研预算压力。

因此，构建一个拥有 Scanpy/Seurat 等平台的数据结构规范，同时具备 Julia 本地性能优势的单细胞分析库，不仅能够提升分析效率，更将有助于推动大规模、深层次的生命科学研究向前迈进，为多模态组学、空间转录组、个体化医疗等新兴方向奠定计算基础。

1.2.2 本项目的目标

本项目的主要目标是用 julia 语言实现一个由 scanpy 启发的数据分析库，用于 scRNA-seq 的分析，弥补 julia 生态在单细胞分析上的不足。

在项目稳步推进并完成一个可用版本的前提下，利用 Julia 在数值计算和并行计算方面的优势，保证 api 的高效性和性能，以实现对大规模数据集的分析工作。API 的设计应符合 julia 官方要求，并尽量与 python 保持一致，本提供完整的手册与官方文档，以降低用户从 scanpy 迁移到 Juscan.jl 的学习成本。另外构建过程采用模块化设计并与 Julia 生态中的其他

工具无缝对接，在提高代码的可读性的同时方便未来的迭代并集成更多高级功能。

第二章 系统设计与实现

2.1 开发环境与工具链

2.1.1 编程语言

Juscan.jl 库使用 julia (version1.11.3)

2.1.2 依赖库

- **DataFrames.jl**: Julia 基础数据框支持
- **HDF5.jl**: 提供 HDF5 数据读写操作
- **LinearAlgebra.jl**: Julia 矩阵和线性代数库
- **Loess.jl**: 回归平滑算法支持
- **Muon.jl**: Muon.Anndata 数据结构及基础行为
- **SparseArrays.jl**: Julia 稀疏矩阵支持
- **Statistics.jl, StatsBase.jl**: 基础统计学函数
- **UMAP.jl**: 提供 UMAP 数据降维函数

2.1.3 开发工具

- CPU: Intel i5-12500
- GPU: NVIDIA GeForce RTX 3050
- Platform: Arch Linux
- 单元测试: **Test.jl**
- 调试器: **Debugger.jl**
- 文档生成: **Documenter.jl**

2.2 Juscan.jl 总体架构设计

2.2.1 整体流程图

如图2.1所示, Juscan.jl 的整体流程图展示了数据分析的主要步骤。首先, 用户通过 **read_h5ad** 函数读取 HDF5 格式的 AnnData 数据集。接着, 数据预处理模块会计算质量控制指标, 并根据用户设定的阈值进行细胞和基因过滤。随后, 数据将被归一化处理, 以消除测序深度差异带来的偏倚。最后, 用户可以选择降维方法 (如 PCA 或 UMAP) 进行数据可视化, 并

使用聚类算法（如 Louvain 或 k-means）对细胞进行分群。

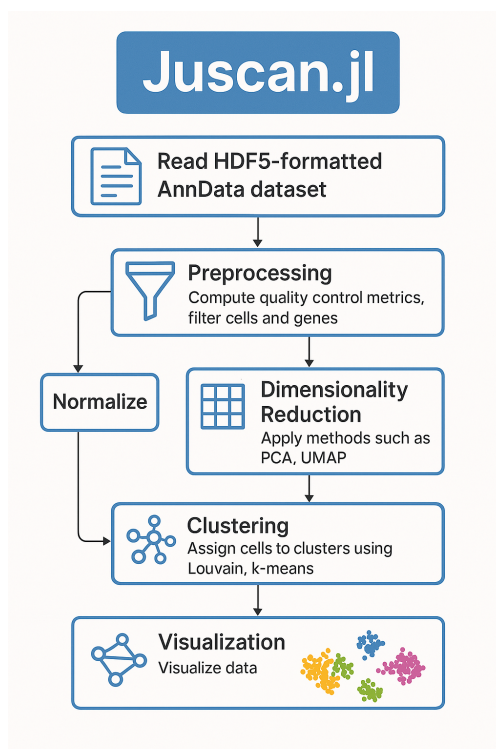


图 2.1 Juscan.jl 流程图

2.2.2 系统模块划分

如代码1，此平台使用模块化的设计，将不同的功能划分到各自所属的模块中，以提高代码的整洁性、可维护性和扩展能力。

Code 1: 文件结构

```

Juscan.jl/
├── src/
│   ├── preprocessing/
│   │   ├── filter.jl           # filter cells and genes
│   │   ├── normalization.jl   # data normalization methods
│   │   ├── pp.jl              # preprocessing module
│   │   ├── qc.jl              # quality control methods
│   │   └── utils.jl
│   └── tools/
│       ├── cluster.jl         # clustering methods
│       ├── hvg.jl             # highly variable genes
│       ├── louvain.jl         # louvain clustering
│       ├── modularityClustering.jl # modularity clustering
│       ├── pca.jl             # dimensionality reduction
│       └── snn.jl             # SNN neighbour
    
```

```

├── tl.jl                                # tools
├── plots/
│   ├── colors.jl
│   ├── pl.jl
│   └── plots.jl
├── Juscan.jl                            # Main module file
├── anndata.jl                           # anndata utilities
├── utils.jl
├── docs/                                # documentation directory
├── test/                                # unit test directory
├── LICENSE
├── Project.toml                         # Julia project files
└── README.md

```

整体的 Juscan.jl 主要分为文档文件夹，单元测试文件夹，以及源代码文件夹。

源代码文件夹中 Juscan.jl 为整个项目的主模块，为项目导入需要的依赖库，加载子模块，以及对外暴露 API 接口。

导入的子模块包括 preprocessing, tools, plots, 分别为单细胞的分析提供预处理，主要分析算法与可视化。

本项目由 Muon.jl 提供官方的 AnnData 数据结构，代码 2 代表了 Muon.anndata 基本结构。

2.3 数据结构

Code 2: Muon.AnnData

```

1 mutable struct AnnData <: AbstractAnnData
2     file::Union{HDF5.File, HDF5.Group, Nothing}
3
4     X::Union{AbstractMatrix{<:Number}, Nothing}
5
6     obs::DataFrame
7     obs_names::Index{<:AbstractString}
8
9     var::DataFrame
10    var_names::Index{<:AbstractString}
11
12    obsm::StrAlignedMapping{Tuple{1 => 1}, AnnData}
13    obsp::StrAlignedMapping{Tuple{1 => 1, 2 => 1}, AnnData}
14
15    varm::StrAlignedMapping{Tuple{1 => 2}, AnnData}
16    varp::StrAlignedMapping{Tuple{1 => 2, 2 => 2}, AnnData}
17

```

```
18   layers::AbstractAlignedMapping{Tuple{1 => 1, 2 => 2}, String}
19
20   uns::Dict{<:AbstractString, <:Any}
21 end
```

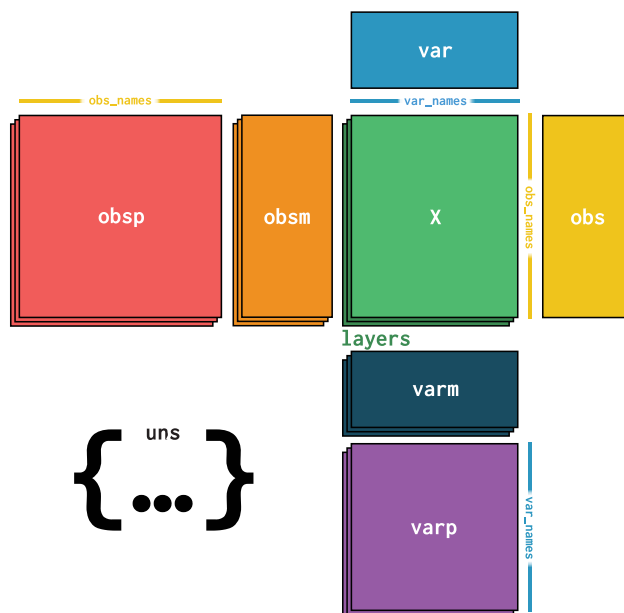


图 2.2 anndata 数据结构示意图

但是 Muon.jl 并没有提供足够多的处理 `anndata` 数据结构的方法行为，为了补充这方面的不足，本项目在 `src/anndata.jl` 文件中进行了一些简单的扩充，包括子集化，插入观测与特征数据框，获取与设观测值的各个维度等方法。

2.4 数据预处理模块

2.4.1 计算质量控制指标

为了评估单细胞或多组学数据中的细胞与基因质量，我们实现了一个模块化的质量控制（quality control, QC）指标计算函数 `calculate_qc_metrics`。该函数支持从不同表达矩阵源（如主矩阵、原始计数矩阵 `adata.raw.X` 或指定的 `layer`）提取表达数据，并分别计算细胞和基因层面的多种常用 QC 指标。

该函数的调用格式如下:

Code 3: Muon.Pp.calculateQcMetrix

```
1 calculate_qc_metrics(  
2     adata;  
3     expr_type="counts",
```

```

4   var_type="genes",
5   qc_vars=String[],
6   percent_top=[50, 100, 200, 500],
7   layer=nothing,
8   use_raw=false,
9   use_log1p=true,
10  parallel=nothing
11 )::Tuple{DataFrame, DataFrame}

```

在细胞维度上，函数会统计每个细胞中被检测到的特征数量、总表达量、log 转换后的表达水平、以及前若干高表达基因（如前 50、100、200 和 500 个）所占的表达比例。此外，用户可通过 `qc_vars` 参数指定一组具有特殊生物学意义的基因类别（如线粒体基因、核糖体基因等），函数将进一步评估这些类别基因在各细胞中的表达总量和比例。

在基因维度上，函数则计算每个基因在细胞群体中的检测频率、平均表达水平、零表达比例（dropout rate）以及总表达量等指标。这些指标可用于后续的基因过滤与特征选择。

函数 `calculate_qc_metrics` 返回两个 `DataFrame` 对象，分别包含细胞与基因层面的 QC 指标；而对应的 `calculate_qc_metrics!` 则为原地版本，直接将计算结果存储于 `adata.obs` 与 `adata.var` 字段中，便于与后续分析模块协同使用。

为支持灵活的数据接口设计，该模块还实现了表达矩阵的抽象选择函数 `_choose_mtx_rep`，并对核心 QC 计算逻辑进行了进一步拆分，分别实现了用于细胞维度的 `describe_obs`/`describe_obs!` 和基因维度的 `describe_var`/`describe_var!`。其中，top-n 表达占比的计算由辅助函数 `top_segment_proportions` 实现。

该模块参考了 Scanpy 与 Seurat 的 QC 实践流程，并结合 Julia 类型系统设计了更高的通用性与可组合性，便于集成到多模态数据分析管线中。

2.4.2 过滤

在质量控制指标计算之后，我们进一步对数据进行细胞与基因的过滤，以剔除潜在的低质量条目，减少技术噪声对下游分析的干扰。本研究实现了一套通用的过滤函数，支持以总表达量、检测到的基因数（或细胞数）为阈值，灵活筛选细胞或基因，如代码4。

Code 4: Muon.Pp.filter

```

1  filter_cells!(
2    data::Muon.AnnData;
3    min_counts=nothing, min_genes=nothing,
4    max_counts=nothing, max_genes=nothing
5  )::Nothing
6  filter_genes!(

```

```

7   data::Muon.AnnData;
8   min_counts=nothing, min_cells=nothing,
9   max_counts=nothing, max_cells=nothing
10 )::Nothing

```

细胞过滤函数 **filter_cells** 支持以下常用策略：

- 最小/最大总表达量 (**min_counts**, **max_counts**)
- 最小/最大检测基因数 (**min_genes**, **max_genes**)
- 可选择返回布尔向量（用于掩码子集）或过滤后的新对象（通过设置 **copy=true**）
- 同时提供 **filter_cells!** 实现原地修改。

类似地，基因过滤函数 **filter_genes** 支持以下标准筛选：

- 被表达的细胞数量阈值 (**min_cells**, **max_cells**)
- 总表达量阈值 (**min_counts**, **max_counts**)
- 同样支持返回掩码或直接修改 **AnnData** 对象

为确保操作语义明确，所有函数均要求每次仅传入一个过滤标准，以避免混淆。被筛选出的条目数量将在控制台输出日志，便于用户了解过滤效果。

该模块基于 **Juscan.subset_adata!** 接口统一实现子集化操作，保证了代码的可维护性与高一致性，可无缝集成至数据预处理 workflows 中。

2.4.3 归一化

为消除不同细胞测序深度（即总 UMI 数）差异所带来的偏倚，我们实现了一个可扩展的总表达量归一化函数 **normalize_total**，用于将每个细胞的表达矩阵缩放至统一的目标总量。该方法类似于 Scanpy 中的 **normalize_total** 或 Seurat 中的 **NormalizeData**，但更具灵活性与模块化。

Code 5: Muon.Pp.normalize

```

1  normalize_total(
2      adata::AnnData;
3      target_sum::Union{Real, Nothing}=nothing,
4      exclude_highly_expressed::Bool=false,
5      max_fraction::Float64=0.05,
6      key_added::Union{String, Nothing}=nothing,
7      layer::Union{String, Nothing}=nothing,
8      layers::Union{String, Vector{String}, Nothing}=nothing,
9      layer_norm::Union{String, Nothing}=nothing,
10     copy::Bool=false
11 )::Union{AnnData, Dict{String, Any}}
12 normalize_total!(

```

```

13     adata::AnnData;
14     target_sum::Union{Real, Nothing}=nothing,
15     exclude_highly_expressed::Bool=false,
16     max_fraction::Float64=0.05,
17     key_added::Union{String, Nothing}=nothing,
18     layer::Union{String, Nothing}=nothing,
19     layers::Union{String, Vector{String}, Nothing}=nothing
20 )::Nothing

```

该函数通过计算每个细胞的总表达量（按行求和）作为归一化因子，并将表达矩阵按比例缩放，从而实现标准化。若未指定目标值（`target_sum`），则默认将每个细胞归一化至群体中位总表达量。此外，函数允许用户通过参数 `exclude_highly_expressed` 排除在某些细胞中高度表达的基因（如线粒体或核糖体基因），避免其对归一化因子的主导效应。

我们提供了两种版本的归一化函数：

`normalize_total`：返回归一化后的新对象或结果字典；

`normalize_total!`：原地（in-place）修改原始数据结构，节省内存。

这两个函数均支持对单个 `layer` 或多个 `layers` 进行归一化操作，并可通过参数 `layer_norm` 控制不同层归一化的参考因子。此外，归一化因子也可通过 `key_added` 参数保存至 `adata.obs` 中，用于后续批量分析或可视化。

值得注意的是，对于表达全为零的细胞，函数会发出警告提示。我们建议在归一化之前结合质量控制结果，先剔除此类无信息细胞。

2.5 工具模块

2.5.1 高可变基因

在单细胞 RNA 测序分析中，高变异基因（highly variable genes, HVGs）的识别对于后续的降维、聚类与轨迹推断等分析具有重要意义。为此，我们基于 Seurat v3 的策略，在 Julia 中实现了一套高变基因筛选模块 `highly_variable_genes`，该方法兼容批次（batch-aware）处理，适用于多样本或多组学场景下的高变基因筛选。

该方法首先对每个批次独立计算每个基因的平均表达量与方差，并通过 LOESS 回归（局部加权多项式拟合）建模均值-方差关系，获得期望方差。随后，对原始方差进行校正，计算归一化后的方差作为变异性的衡量指标。对于跨多个批次的分析，方法进一步对各批次筛选结果进行合并，通过多批次中位排名的方式选出最终的一组 HVGs，保证其在多个批次中稳定地表现出高变异性。

我们实现了以下三类函数接口：

Code 6: Muon.Tl.hvg

```

1 subset_to_hvg!(adata::AnnData;
2   layer::Union{String,Nothing} = nothing,
3   n_top_genes::Int=2000,
4   batch_key::Union{String,Nothing} = nothing,
5   span::Float64=0.3,
6   verbose::Bool=true
7 )
8 highly_variable_genes(adata::AnnData;
9   layer::Union{String,Nothing} = nothing,
10  n_top_genes::Int=2000,
11  batch_key::Union{String,Nothing} = nothing,
12  span::Float64=0.3
13 )
14 highly_variable_genes!(adata::AnnData;
15   layer::Union{String,Nothing} = nothing,
16   n_top_genes::Int=2000,
17   batch_key::Union{String,Nothing} = nothing,
18   span::Float64=0.3,
19   replace_hvgs::Bool=true,
20   verbose::Bool=false
21 )

```

函数输出包括每个基因的均值、方差、归一化方差、在多个批次中的高变频次，以及最终的变异性排名。若数据中已有 HVG 注释，用户可选择是否替换（`replace_hvgs=true`）。

2.5.2 降维

我们首先实现了 `log_transform!` 与 `logp1_transform!` 函数，用于对数据执行 $\log(x + \varepsilon)$ 或 $\log(1+x)$ 变换，以增强表达量在低值区间的分布分辨能力。默认会将结果保存至新的数据层，便于复用与回溯。此外，函数 `standardize` 与 `prcomps` 封装了标准化与主成分提取过程，提供更底层的控制接口。

(1)pca

函数 `pca!` 实现了对 `AnnData` 对象中指定数据层的 PCA 计算，并将结果存储于 `adata.obsm["PCA"]` 中，供后续分析调用。该函数支持自动寻找合适的输入表达矩阵：若未提供目标数据层，则会依次查找是否存在经过对数转换的表达矩阵（`"log_transformed"`）或归一化数据层（`"normalized"`）；若均不存在，则自动执行归一化与对数转换流程。

Code 7: Muon.Tl.pca

```

1  pca!(
2      adata::Muon.AnnData;
3      layer::String="log_transformed",
4      n_pcs::Int=1000,
5      verbose::Bool=true
6  )

```

为确保主成分数量的合理性，函数会根据数据矩阵的维度自动调整主成分数量（`n_pcs`），避免维度溢出问题。计算过程采用稠密矩阵的奇异值分解（SVD）或稀疏矩阵的截断 SVD（TSVD），以兼容不同规模数据。

(2)umap

Code 8: Muon.Tl.umap

```

1  umap!(
2      adata::Muon.AnnData;
3      layer::String="log_transformed",
4      use_pca_init::Bool=false,
5      n_pcs::Int=100,
6      verbose::Bool=true,
7      kwargs...
8  )

```

函数 `umap!` 用于计算数据的 UMAP 嵌入，并将结果包括嵌入坐标、最近邻索引、距离矩阵与模糊邻接图分别存入 `adata.obsm` 与 `adata.obsp` 中。该函数支持两种初始化方式：

- 直接嵌入：使用指定数据层或自动生成的对数转换数据
- PCA 初始化：通过设置 `use_pca_init=true`，先执行 PCA 作为初始嵌入向量

UMAP 本身使用 `UMAP.jl` 实现，默认基于余弦距离构建邻接图，并可通过传入关键字参数（如 `n_neighbors`, `min_dist` 等）实现参数化控制。

2.5.3 聚类

聚类分析是单细胞数据中识别细胞亚群结构的关键步骤。此部分代码部分参考 `ASCT.jl` 的实现方法^[15]，对 `Muon: AnnData` 数据结构进行两类聚类方法：图结构驱动的模式度聚类（modularity clustering）与基于中心点的 KMedoids 聚类（近似 KMeans），构成了双路径的细胞聚类框架。为了便于用户在不同分析阶段调用聚类方法，我们提供了统一接口 `clustering!`，支持多种参数配置与算法选择。

Code 9: Muon.Tl.Clustering

```

1 function clustering!(
2     data::Muon.AnnData;
3     method::AbstractString="mc",
4     reduction::Union{AbstractString, Symbol}=:auto,
5     use_pca::Union{AbstractString, Integer}="pca_cut",
6     # n_pcs::Union{Integer, AbstractString}=5,
7     tree_K::Integer=20,
8     resolution::Union{Symbol, Real, AbstractRange}=:auto,
9     cluster_K::Union{Nothing, Integer}=nothing,
10    cluster_K_max::Union{Nothing, Integer}=30,
11    dist::AbstractString="Euclidean",
12    network::AbstractString="SNN",
13    random_starts_number::Integer=10,
14    iter_number::Integer=10,
15    prune::AbstractFloat=1 / 15,
16    seed::Integer=-1,
17 )

```

对AnnData对象执行聚类分析,结果会自动保存在adata.obs中。该函数根据参数method选择不同的聚类策略。

a. 基于模块度的图聚类 (modularity clustering)

该方法通过构建细胞的共享最近邻图 (Shared Nearest Neighbor, SNN), 并以最大化模块度 (modularity) 为目标, 对细胞图进行社团划分。方法流程如下:

1. 图构建: 在降维空间 (如 PCA) 中, 使用 KD 树加速寻找每个细胞的 K 个近邻, 构建 KNN 或 SNN 图。SNN 图定义如下:

$$\text{SNN}_{ij} = |\mathcal{N}_i \cap \mathcal{N}_j| / K \quad (4)$$

其中 $\mathcal{N}_i$ 表示细胞 i 的邻居集合。

2. 图稀疏化与剪枝: 通过阈值参数 $\alpha \in (0, 1]$ 对图进行稀疏化, 仅保留重叠度高的邻接关系, 以提高模块度检测的稳定性。
3. 模块度优化聚类: 通过 Louvain 或 Leiden 等算法, 在图中最大化以下模块度函数:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \quad (5)$$

其中 A_{ij} 为图邻接矩阵, k_i 为节点度, γ 为分辨率参数, δ 为 Kronecker delta。我们支持不同的 γ 参数进行多分辨率扫描, 自动推荐最优分辨率。

b. 基于 KMedoids 的中心点聚类 (k-medoids clustering)

KMedoids 聚类是一种鲁棒于异常值的变体,相较于 KMeans 选择“簇中心”,KMedoids 选择实际数据点作为中心。我们实现如下自动聚类过程:

1. 距离计算: 用户可选择多种距离度量,如欧氏距离、余弦距离、相关距离等,在降维空间中计算细胞间距离矩阵。
2. 自动选择簇数 K :
 - 计算多个 K 值下的聚类结果 (并行加速);
 - 对每个聚类结果,计算 silhouette 系数 $\bar{s}$ 与标准差;
 - 同时评估总成本的肘部法则 (elbow method);
 - 综合两个指标,自动选择最优 K 值:

$$K^* = \arg \max_K (\text{silhouette score} - \text{total cost deviation}) \quad (6)$$

2.6 可视化模块

单细胞组学分析常涉及高维数据的降维、聚类和分类结果,为了更好地呈现这些复杂结构,我们在 `Juscan.Plots` 中实现了一套基于 `CairoMakie` 的绘图模块,涵盖多种用于探索性分析与结果展示的图形函数。该模块强调高可配置性、视觉一致性与图像质量,便于科研报告与论文作图。

2.6.1 调色机制与主题色板

我们设计了一套模块化的调色系统,通过 `get_palette` 和 `get_continuous_colormap` 接口,提供一系列离散与连续的调色方案,支持自动扩展与下采样。其中,离散色板如 `"friendly"`、`"apple"`、`"rainbow"` 等适用于类别标签着色,而连续色带则集成自 `ColorSchemes.jl`,可用于表达量、主成分值等数值信息的可视化。所有色板均通过自定义函数实现扩展与裁剪策略,确保图形风格在多图展示中保持一致。

2.6.2 可视化函数接口

模块当前提供以下核心函数:

- `violin`: 绘制提琴图,展示多个观测变量 (如基因表达、细胞指标) 的分布,支持点抖动 (jitter) 与多子图并排布局。
- `scatter`: 绘制二维散点图,支持根据第三变量进行着色 (如根据表达量、UMAP/PCA 位置等变量上色)。
- `hvg_scatter`: 用于展示高变异基因的筛选结果,通过平均表达与变异度关系图突出高变基因在全体中的位置。
- `plot_variance_ratio`: 绘制 PCA 主成分的方差解释比例图,支持对前若干主成分进行可视化,并自动标注前 5 个 PC。

- **plot_umap**: 绘制 UMAP 降维嵌入图, 支持按照类别标签上色, 自动生成图例并匹配色板风格。

(1) violin: 分布可视化

该函数支持绘制多个变量在样本中的表达分布, 可用于细胞质控指标 (如总 UMI 数、线粒体比例) 的展示。用户可指定色板名与透明度参数, 自动匹配子图数量与调色方案。此外还可设置点抖动强度与保存图像路径:

Code 10: Juscan.Plots.violin

```
1 violin(  
2     adata::AnnData,  
3     keys::Union{String, Vector{String}};  
4     width::Real=600,  
5     height::Real=400,  
6     jitter::Union{Bool, Real}=0.5, dot_size::Real=2,  
7     title::String="violin plot",  
8     palette_name::String="friendly",  
9     fill_alpha::Real=1.0,  
10    savefig::Union{Bool, String}=false  
11 )
```

(2) hvg_scatter: 高变基因展示

为辅助用户评估 HVG 的筛选结果, 我们实现了 **hvg_scatter** 函数, 绘制均值-方差与归一化方差的二维图。灰色表示非 HVG 基因, 黑色点表示已筛选出的 HVGs, 有助于观察其在全体中的分布位置。

(3) plot_umap: 嵌入可视化

该函数根据 **adata.obsm** 中的 UMAP 坐标绘图, 并根据 **adata.obs** 中的分类变量进行着色。支持自定义调色板、图形尺寸与图像保存路径:

Code 11: Juscan.Plots.plot umap

```
1 plot_umap(  
2     adata;  
3     color_by::String="clusters_latest",  
4     key="umap",  
5     palette_name::String="rainbow",  
6     savefig::Union{Bool, String}=false  
7 )
```

(4) scatter 与 plot_variance_ratio

除标准二维散点图外,我们也实现了用于主成分方差比分析的函数 `plot_variance_ratio`。其通过绘制 log 方差比图,帮助用户直观判断 PCA 的有效维度。

第三章 实验与评估

3.1 实验数据集介绍

本研究使用的单细胞转录组数据集为 10x Genomics 公司公开发布的经典示例数据集——PBMC 3k 数据集。该数据集包含约 2,700 个外周血单核细胞 (Peripheral Blood Mononuclear Cells, PBMC)，来自一位健康成年供体。PBMC 包括 T 细胞、B 细胞、自然杀伤细胞和单核细胞等多种免疫相关细胞类型，常作为免疫学研究中的模型系统。

PBMC 3k 数据集采用 10x Genomics Chromium 单细胞 3' RNA 测序平台进行文库构建，并使用 Illumina NextSeq 500 高通量测序仪进行测序。每个细胞平均测序深度约为 69,000 个 reads。原始数据使用 10x Genomics 官方分析工具 Cell Ranger (版本 1.1.0) 进行预处理，包括细胞条形码识别、UMI 去重及基因表达矩阵生成。数据结果以稀疏矩阵形式存储，包含每个细胞在各个基因上的表达计数。

本数据集可以在 10X Genomics 官网获取，并可以使用 SeuratDisk, scDior 等工具转换为 h5ad 文件。

由于数据来源权威、质量可靠、样本复杂性适中，PBMC 3k 数据集已广泛应用于 Seurat、Scanpy 等单细胞分析工具的功能验证与流程教学中，是验证聚类、降维、批次效应处理等算法性能的标准数据集之一。

3.2 实验设计与流程

本文基于 PBMC 3k 数据集，参考官方流程，实现了单细胞 RNA 测序数据分析的完整流程。以下是使用 Juscan.jl 库进行数据分析的示例：

3.2.1 数据读取与质量控制

Juscan.jl 库使用 Muon.jl 库作为数据存储的基础，支持读取 h5ad 格式的数据文件。数据读取后，使用 `Juscan.Pp.filter_cells!` 和 `Juscan.Pp.filter_genes!` 函数进行细胞和基因的过滤，去除低质量细胞和低表达基因。接着，计算线粒体基因、核糖体基因和血红蛋白基因的表达比例，并根据这些指标进行进一步的质量控制。

Code 1: Juscan.jl demo data load and quality control

```
1 using Juscan, Muon
2 using DataFrames, LinearAlgebra, SparseArrays
3
```

```

4  adata = Juscan.readh5ad("./data/pbmc_3k.h5ad")
5
6  ##### quality control #####
7
8  Juscan.Pp.filter_cells!(adata, min_genes=200)
9  Juscan.Pp.filter_genes!(adata, min_cells=3)
10 Juscan.Pp.filter_cells!(adata, max_genes=2300)
11 Juscan.Pp.filter_cells!(adata, max_counts=10000)
12
13 adata.var.mt = startswith.(adata.var_names, "MT-")
14 adata.var.ribo = startswith.(adata.var_names, "RPS") .| startswith.(adata.
    var_names, "RPL")
15 adata.var.hb = occursin.(r"^HB[^P]", adata.var_names)
16
17 Juscan.Pp.calculate_qc_metrics!(adata, qc_vars=["mt", "ribo", "hb"])
18 adata = adata[adata.obs[:, "pct_counts_mt"] .< 8, :]
19 adata = adata[adata.obs[:, "pct_counts_mt"] .> 0.5, :]
20
21 Juscan.Pl.violin(
22     adata,
23     ["pct_counts_mt", "n_genes_by_counts", "total_counts"];
24     width=300,
25     height=800,
26     fill_alpha=1,
27     savefig="/home/lihuax/Pictures/Juscan/qc_violin.png",
28 )
29
30 Juscan.Pl.scatter(
31     adata,
32     "total_counts",
33     "n_genes_by_counts",
34     color_key="pct_counts_mt",
35     width=800,
36     height=800,
37     colormap_name="magma",
38     savefig="/home/lihuax/Pictures/Juscan/qc_scatter.png",
39 )

```

运行上述代码后，获得处理后的高质量细胞数据集。为了直观展示质量控制的效果，使用小提琴图用于展示线粒体比例、总转录本数量与基因数的分布情况，帮助我们观察数据的整体质量分布；而散点图则揭示了转录本总数与检测到的基因数之间的关系，并以线粒体比例着色，从而更好地识别潜在的异常细胞。

图 3.1 所示即为本次质量控制过程中生成的 QC 图像。

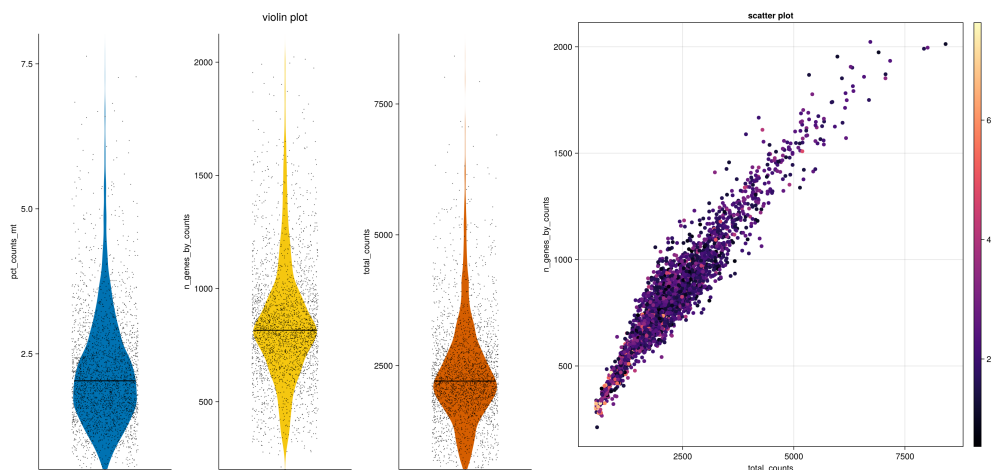


图 3.1 Juscan.jl 质量控制可视化结果

3.2.2 数据归一化与高变基因

首先，使用 `Juscan.Pp.normalize_total!` 函数对每个细胞的转录本数进行归一化，使其总表达量一致（如 1000），从而消除不同细胞间测序深度的影响。随后，应用 `Juscan.Tl.logp1_transform!` 函数对归一化后的数据进行对数变换 ($\log(x+1)\log(x+1)$)，以缓解表达量跨度过大的问题，使其更符合后续线性模型的假设。

Code 2: Juscan.jl normalize and find hvg

```
1 ##### normalization #####
2 adata.layers["normalized"] = deepcopy(adata.X)
3 Juscan.Pp.normalize_total!(adata, target_sum=1000, layer="normalized")
4 Juscan.Tl.logp1_transform!(adata, layer="normalized", key_added="
    normalized_logp1")
5 adata.layers["normalized_logp1"] = Float64.(adata.layers["normalized_logp1"])
6
7 ##### highly variable genes #####
8 Juscan.Tl.highly_variable_genes!(adata, n_top_genes=2000, layer="
    normalized_logp1")
9 Juscan.Pl.hvg_scatter(adata, savefig="/home/lihuax/Pictures/Juscan/
    hvg_scatter.png")
```

在归一化的基础上，使用 `Juscan.Tl.highly_variable_genes!` 方法从全基因集中筛选出表达波动性最强的 2000 个基因。这些基因包含了最丰富的生物学信号，是降维与聚类分析的核心输入。

图 3.2 展示了 PCA 肘部图和高变基因的分布图，肘部图用于后续分析中选择 pca 维度数字而 hvg 图中显示了高变基因的分布，其中深色点为筛选出的高变基因。

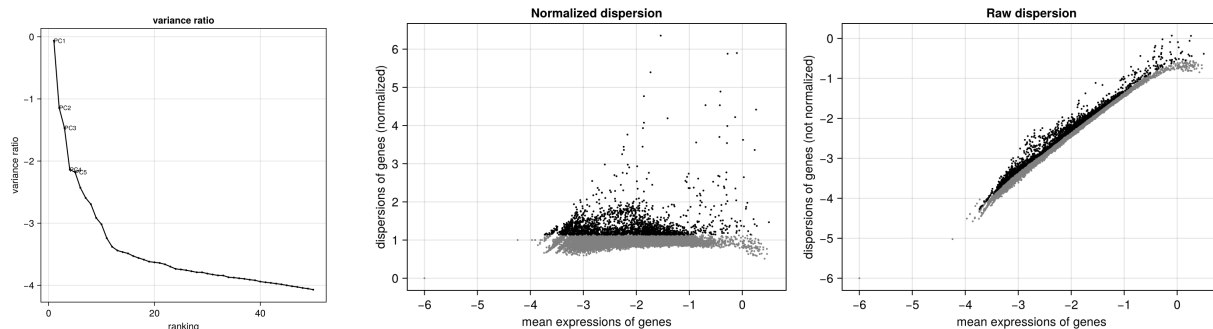


图 3.2 Juscan.jl 归一化和高变基因可视化结果

3.2.3 降维与聚类

在完成归一化与高变基因筛选之后，进行高维数据的压缩与结构发现。首先，通过 `Juscan.Tl.subset_to_hvg!` 筛选前 1000 个高变基因，以提取最具代表性的表达信号。随后，我们使用 `Juscan.Tl.pca!` 执行主成分分析（PCA），将原始维度浓缩为若干主成分，同时保存方差贡献比，便于选择合适的维度数进行后续分析。

紧接着，我们通过社区发现算法对 PCA 空间中的细胞进行聚类，以发现细胞的潜在亚群结构。通过 `Juscan.Tl.umap!` 函数进一步将 PCA 结果降维至二维空间，用于可视化聚类结果。

Code 3: Juscan.jl dimensionality reduction and clustering

```
1 ##### dimensionality reduction #####
2 Juscan.Tl.subset_to_hvg!(adata; layer="normalized_logp2", n_top_genes=1000)
3
4 Juscan.Tl.pca!(adata; layer="normalized_logp1", key_added="pca", n_pcs=50)
5 Juscan.Pl.plot_variance_ratio(adata, savefig="/home/lihuax/Pictures/Juscan/
   variance_ratio.png")
6
7 ##### clustering #####
8 Juscan.Tl.clustering!(adata, reduction="pca", use_pca=10, resolution=0.6)
9 Juscan.Tl.umap!(
10     adata;
11     key_added="umap",
12     use_pca="pca",
13     n_pcs=10,
14     min_dist=0.5,
15     n_neighbors=30,
16 )
17 Juscan.Pl.plot_umap(
18     adata,
19     color_by="clusters_latest",
```

```
20 key="umap",
21 savefig="/home/lihuax/Pictures/Juscan/clusters.png",
22 )
```

图 3.3 展示了 UMAP 嵌入空间中的聚类结果。每个点代表一个细胞，颜色区分不同的聚类标签。聚类结果的清晰分离，提示着数据中存在明显的生物学异质性，可能对应于不同类型的免疫细胞或功能状态。通过这样的降维可视化图，我们不仅能够观察整体结构，还能识别边缘群体与潜在的亚群细胞，为后续的注释与生物学解释奠定坚实基础。

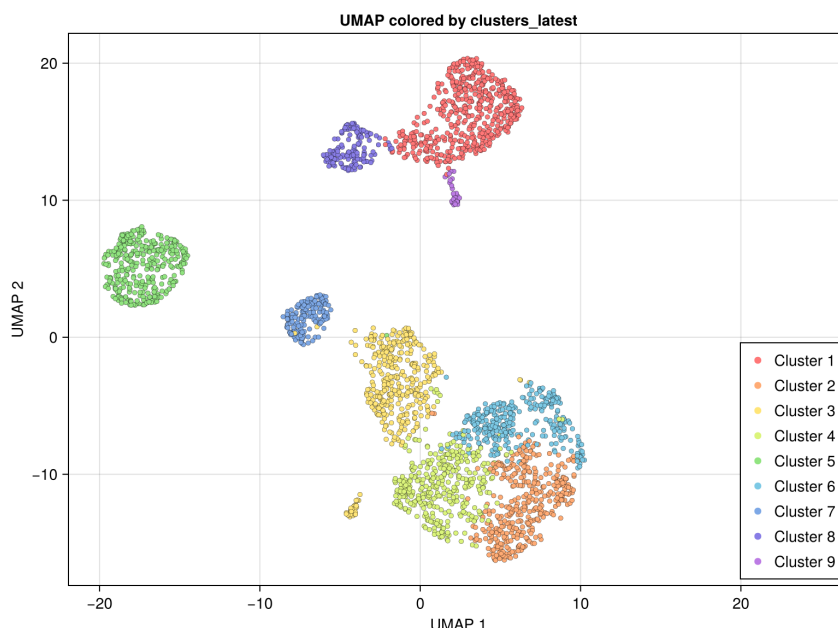


图 3.3 Juscan.jl 聚类结果 umap 可视化

3.3 性能评估

为了全面评估不同语言实现对单细胞 RNA 测序数据处理流程的效率与性能差异，本文基于 PBMC 3k 数据集，分别使用 Julia、Python 与 R 三种主流语言构建了统一流程的 benchmark 测试系统。三种实现版本在逻辑结构、分析步骤及参数设置上保持一致，涵盖典型的单细胞分析流程，包括质量控制、归一化处理、高变基因选择、主成分分析（PCA）、聚类分析与 UMAP 可视化。

其中，Julia 的测试基于 **BenchmarkTools.jl** 库，python 和 R 分别使用 **time** 模块和 **system.time** 方法自行构建性能测试函数。在每个步骤中，实验统一采用 100 次重复采样方式对每个函数进行性能测量，并最终输出时间与内存占用的统计汇总结果（CSV 格式）。

详细的测试代码请见附件。

3.3.1 Seurat 对比 juscan.jl 评估结果

如下图3.4是对比 Seurat 和 juscan.jl 的性能评估结果。

由于 Seurat 和 Juscan 的逻辑差异比较大，filter 和计算质量控制矩阵步骤没有被单独封装为函数，因此这里并没比较这些步骤的效率。

而对于已经比较的步骤，Juscan 的效率明显优于 Seurat，尤其在归一化，降维和聚类方面，Juscan 的性能优势更为明显。

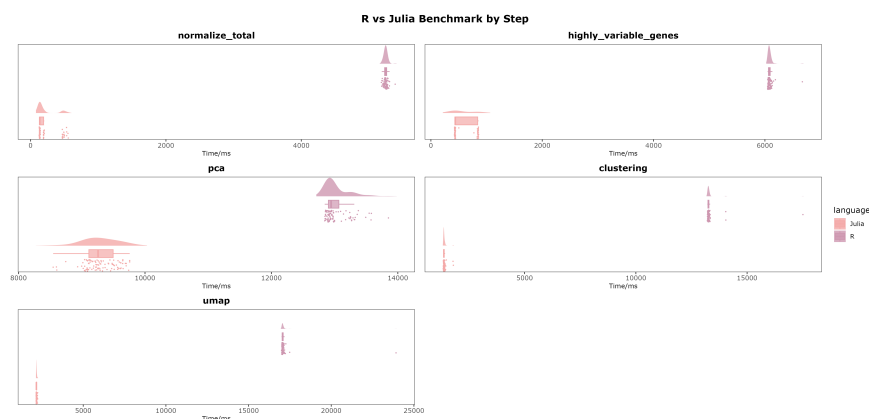


图 3.4 Juscan.jl 与 Seurat 性能对比

3.3.2 scanpy 对比 juscan.jl 评估结果

如下图3.5是对比 scanpy 和 juscan.jl 的性能评估结果。

可以明显看到，python 在数据读取、归一化、PCA 和 UMAP 等步骤上均表现出显著的性能优势，尤其在降维方面。另外 Juscan 的性能表现并不稳定，在不同的测试中，性能波动较大，可能与 Julia 的 JIT 编译机制有关。

总之，在归一化和降维，聚类步骤上，Python 的性能明显优于 Julia，Juscan.jl 任有恒大的进步空间。

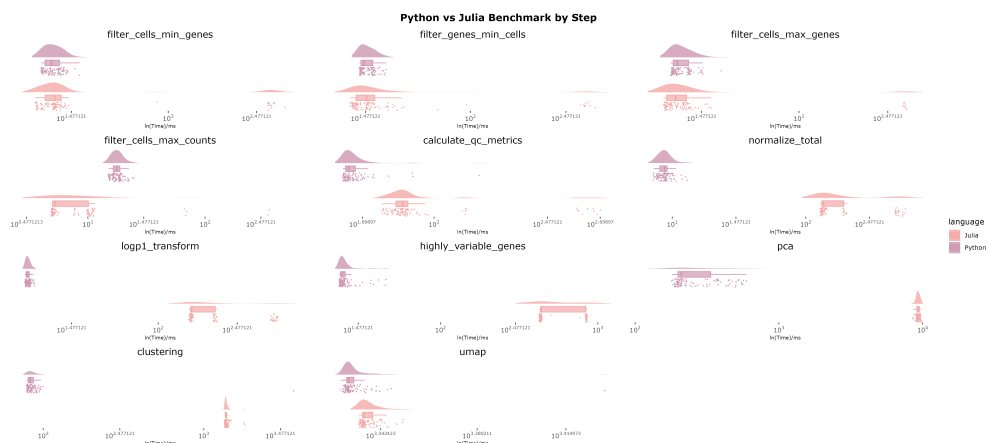


图 3.5 Juscan.jl 与 Scanpy 性能对比

第四章 讨论

4.1 研究成果总结

本研究基于 Julia 编程语言成功设计并实现了一个高性能的单细胞数据分析平台——Juscan.jl。该平台以 Muon.jl 所提供的 AnnData 数据结构为核心，复刻并优化了 Scanpy 在 Python 生态中的分析流程，完成了从数据质量控制、标准化、高变基因筛选、降维、聚类到可视化的完整管线，构建出一个功能完备、接口友好、计算高效的分析框架。

更重要的是，Juscan.jl 采用模块化设计，具有良好的可扩展性和可维护性，为后续的算法开发和功能拓展提供了坚实的基础。

4.2 未来工作展望

尽管 Juscan.jl 已实现了基础的单细胞分析功能，但作为一个初步探索性的项目，仍存在许多可提升的方向，具体包括以下几个方面：

1. 多模态数据支持与整合: 当前版本主要面向 scRNA-seq 数据, 未来可进一步支持 ATAC-seq、CITE-seq 以及空间转录组等多组学数据, 并结合 Canonical Correlation Analysis、WNN 等多模态聚类方法, 实现更全面的细胞表征。
2. 更多聚类与轨迹推断算法的集成: 目前聚类模块仅支持 Louvain 与 k-medoids, 后续可引入 Leiden 算法、基于高斯混合模型 (GMM) 的聚类方法, 甚至构建基于图卷积神经网络 (GCN) 的聚类与分类器。同时, 可引入 pseudotime 轨迹推断模块, 满足发育过程建模的需求。
3. 计算性能的进一步优化: 尽管 Julia 语言本身具备极高的运行性能, 但本项目中的部分性能瓶颈主要源于代码层面的实现尚不够成熟。例如, 部分函数存在数据复制冗余、内存访问不充分优化、缺乏懒计算等问题, 影响了整体的执行效率与稳定性。未来可以通过进一步剖析关键函数的瓶颈 (如使用 @profile、@btime 等工具)、重构核心逻辑、合理安排内存布局、并结合并行/多线程机制等方式, 显著提升 Juscan.jl 的运行效率与工程质量。同时, 也可结合 PackageCompiler.jl 生成系统映像, 以减少启动与冷启动时的性能损耗, 从而带来更流畅的用户体验。
4. 用户交互体验与可视化增强: 当前可视化模块虽已支持 UMAP、提琴图等常规图形, 但仍缺乏交互式浏览器支持。可借助 Makie.jl + Genie.jl 或 Pluto.jl 构建 Web 可视化平台, 让用户通过图形界面进行数据探索。

参考文献

- [1] Poulin J F, Tasic B, Hjerling-Leffler J, et al. Disentangling neural cell diversity using single-cell transcriptomics[J]. *Nature Neuroscience*, 2016, 19:1131–1141.
- [2] Li X, Wang C Y. From bulk, single-cell to spatial rna sequencing[J]. *International Journal of Oral Science*, 2021, 13:36.
- [3] Scanpy –Single-Cell Analysis in Python[Z]. <https://scanpy.readthedocs.io/en/stable/index.html>.
- [4] Zhang S, Li X, Lin J, et al. Review of single-cell rna-seq data clustering for cell-type identification and characterization[J]. *RNA*, 2023, 29(5):517–530.
- [5] Blondel V D, Guillaume J L, Lambiotte R, et al. Fast unfolding of communities in large networks[J]. *Journal of statistical mechanics: theory and experiment*, 2008, 2008(10):P10008.
- [6] Hao Y, Hao S, Andersen-Nissen E, et al. Integrated analysis of multimodal single-cell data[J]. *Cell*, 2021, 184(13):3573–3587.e29. Publisher: Elsevier.
- [7] Hao Y, Stuart T, Kowalski M H, et al. Dictionary learning for integrative, multimodal and scalable single-cell analysis[J]. *Nature Biotechnology*, 2024, 42(2):293–304.
- [8] Wolf F A, Angerer P, Theis F J. SCANPY: large-scale single-cell gene expression data analysis[J]. *Genome Biol*, 2018, 19(1):1–5. Number: 1 Publisher: BioMed Central.
- [9] Bredikhin D, Kats I, Stegle O. MUON: multimodal omics analysis framework[J]. *Genome Biology*, 2022, 23:42.
- [10] Amezquita R A, Lun A T L, Becht E, et al. Orchestrating single-cell analysis with Bioconductor[J]. *Nat Methods*, 2020, 17(2):137–145. Publisher: Nature Publishing Group.
- [11] Sponsor @JuliaLang on GitHub Sponsors[Z]. <https://github.com/sponsors/JuliaLang>.
- [12] Roesch E, Greener J G, MacLean A L, et al. Julia for biologists[J]. *Nat Methods*, 2023, 20(5):655–664.
- [13] Ha M. scvi.jl: A julia package for fitting vaes to single-cell data using count distributions[Z], 2023. <https://github.com/maren-ha/scVI.jl>. Accessed: 2025-04-14.
- [14] Lopez R, Regier J, Cole M B, et al. Deep generative modeling for single-cell transcriptomics[J]. *Nat Methods*, 2018, 15(12):1053–1058. Publisher: Nature Publishing Group.
- [15] Yang L, Li N. Asct: automatic single-cell toolbox in julia[J]. *bioRxiv*, 2023.

附 录

A.1 Juscan.jl 源代码与文档

源码见 Github 仓库: <https://github.com/zehua0417/Juscan.jl.git>

其他更多使用细节见文档: <https://zehua0417.github.io/Juscan.jl/>

A.2 性能检测

Juscan.jl 速度检测

```
1  #!/usr/bin/julia
2
3  # using Pkg
4  # Pkg.add(path="/home/lihuax/Projects/thesis/Juscan.jl")
5  using Juscan
6  using Muon
7  using DataFrames
8  using BenchmarkTools
9  using CSV
10
11  benchmark_times = 100
12  max_time = 1000
13  adata = Juscan.readh5ad("./data/counts.h5ad")
14
15  ##### quality control #####
16  benchmark_filter_cells_min_genes = @benchmark Juscan.Pp.filter_cells!(adata,
    min_genes=100) setup = (adata = Juscan.readh5ad("./data/counts.h5ad"))
    samples = benchmark_times evals = 1 seconds = max_time
17  benchmark_filter_genes_min_cells = @benchmark Juscan.Pp.filter_genes!(adata,
    min_cells=3) setup = (adata = Juscan.readh5ad("./data/counts.h5ad"))
    samples = benchmark_times evals = 1 seconds = max_time
18  benchmark_filter_cells_max_genes = @benchmark Juscan.Pp.filter_cells!(adata,
    max_genes=2000) setup = (adata = Juscan.readh5ad("./data/counts.h5ad"))
    samples = benchmark_times evals = 1 seconds = max_time
19  benchmark_filter_cells_max_counts = @benchmark Juscan.Pp.filter_cells!(adata,
    max_counts=10000) setup = (adata = Juscan.readh5ad("./data/counts.h5ad"))
    samples = benchmark_times evals = 1 seconds = max_time
20
21  Juscan.Pp.filter_cells!(adata, min_genes=100)
22  Juscan.Pp.filter_genes!(adata, min_cells=3)
```

```

23 Juscan.Pp.filter_cells!(adata, max_genes=2000)
24 Juscan.Pp.filter_cells!(adata, max_counts=10000)
25
26 adata.var.mt = startswith.(adata.var_names, "MT-")
27 adata.var.ribo = startswith.(adata.var_names, "RPS") .| startswith.(adata.
    var_names, "RPL")
28 adata.var.hb = occursin.(r"^HB[^P]", adata.var_names)
29
30 benchmark_qc = @benchmark Juscan.Pp.calculate_qc_metrics!(adata, qc_vars=["mt
    ", "ribo", "hb"]) samples = benchmark_times evals = 1 seconds = max_time
31
32 adata = adata[adata.obs[:, "pct_counts_mt"].<11, :]
33 adata = adata[adata.obs[:, "pct_counts_mt"].>0.5, :]
34
35 Juscan.Pl.violin(
36     adata,
37     ["pct_counts_mt", "n_genes_by_counts", "total_counts"];
38     width=300,
39     height=800,
40     fill_alpha=0.7,
41     savefig="./img/julia/qc_violin.png",
42 )
43
44 Juscan.Pl.scatter(
45     adata,
46     "total_counts",
47     "n_genes_by_counts",
48     color_key="pct_counts_mt",
49     width=800,
50     height=800,
51     colormap_name="magma",
52     savefig="./img/julia/qc_scatter.png",
53 )
54
55 ##### normalization #####
56 benchmark_normalize_total = @benchmark Juscan.Pp.normalize_total!(adata,
    target_sum=1000, layer="normalized") setup = (adata.layers["normalized"]
    = deepcopy(adata.X)) samples = benchmark_times evals = 1 seconds =
    max_time
57 benchmark_normalize_logp1 = @benchmark Juscan.Tl.logp1_transform!(adata,
    layer="normalized", key_added="normalized_logp1") samples =
    benchmark_times evals = 1 seconds = max_time
58
59 ##### highly variable genes #####
60 benchmark_hvg = @benchmark Juscan.Tl.highly_variable_genes!(adata,

```

```

        n_top_genes=2000, layer="normalized_logp1") samples = benchmark_times
        evals = 1 seconds = max_time
61 Juscan.Pl.hvg_scatter(adata, savefig="./img/julia/hvg_scatter.png")
62
63 ##### dimensionality reduction #####
64 # Juscan.Tl.subset_to_hvg!(adata; layer="normalized_logp1", n_top_genes=2000)
65
66 adata.layers["normalized_logp1_backup"] = deepcopy(adata.layers["
        normalized_logp1"])
67 benchmark_pca = @benchmark Juscan.Tl.pca!(adata; layer="normalized_logp1",
        key_added="pca", n_pcs=50) setup = (adata.layers["normalized_logp1"] =
        deepcopy(adata.layers["normalized_logp1"])) samples = benchmark_times
        evals = 1 seconds = max_time
68
69 ##### clustering #####
70 benchmark_clustering_km = @benchmark Juscan.Tl.clustering!(adata, method="km"
        , use_pca=15, cluster_K=6, dist="Euclidean") samples = benchmark_times
        evals = 1 seconds = max_time
71 benchmark_clustering_louvain = @benchmark Juscan.Tl.clustering!(adata,
        reduction="pca", use_pca=5, resolution=0.3) samples = benchmark_times
        evals = 1 seconds = max_time
72
73 benchmark_umap = @benchmark Juscan.Tl.umap!(
74     adata;
75     use_pca="pca",
76     key_added="umap",
77     n_pcs=5,
78     min_dist=0.3,
79     n_neighbors=40,
80 ) samples = benchmark_times evals = 1 seconds = max_time
81 Juscan.Pl.plot_umap(adata, color_by="clusters_latest", savefig="./img/julia/
        umap_without_subset_hvg.png")
82
83
84 ##### benchmark results #####
85
86 function extract_benchmark_summary(name::String, result::BenchmarkTools.Trial
        )
87     return (step=name,
88         time_ns=minimum(result).time,
89         memory_bytes=minimum(result).memory,
90         allocs=minimum(result).allocs)
91 end
92
93 function extract_times_df(name::String, trial::BenchmarkTools.Trial)

```



```

94     times = trial.times
95     return DataFrame(step=fill(name, length(times)), run_id=1:length(times),
96                       time_ns=times)
97
98     benchmarks = [
99         ("filter_cells_min_genes", benchmark_filter_cells_min_genes),
100        ("filter_genes_min_cells", benchmark_filter_genes_min_cells),
101        ("filter_cells_max_genes", benchmark_filter_cells_max_genes),
102        ("filter_cells_max_counts", benchmark_filter_cells_max_counts),
103        ("calculate_qc_metrics", benchmark_qc),
104        ("normalize_total", benchmark_normalize_total),
105        ("logp1_transform", benchmark_normalize_logp1),
106        ("highly_variable_genes", benchmark_hvg),
107        ("pca", benchmark_pca),
108        ("clustering_kmeans", benchmark_clustering_km),
109        ("clustering_louvain", benchmark_clustering_louvain),
110        ("umap", benchmark_umap),
111    ]
112    summary_df = DataFrame([extract_benchmark_summary(name, trial) for (name,
113        trial) in benchmarks])
114
115    # change unit
116    summary_df.time_ms = summary_df.time_ns ./ 1e6
117    summary_df.memory_MB = summary_df.memory_bytes ./ (1024^2)
118    # select!(summary_df, :step, :time_ms, :memory_MB, :alllocs)
119
120    all_times_df = vcat([extract_times_df(name, trial) for (name, trial) in
121        benchmarks]...)
122    all_times_df.time_ms = all_times_df.time_ns ./ 1e6
123
124    # save as csv
125    CSV.write("./result/julia/benchmark_summary.csv", summary_df)
126    CSV.write("./result/julia/benchmark_times.csv", all_times_df)

```

Seurat 速度检测

```

1  #! /home/lihuax/miniconda3/envs/r422/bin/Rscript
2
3  library(rlang)
4  library(dplyr)
5  library(tibble)
6  library(magrittr)
7  library(readr)
8  library(ggplot2)

```

```

9  library(Seurat)
10 library(SeuratDisk)
11 library(pryr)
12
13 benchmark_times=100
14
15 benchmark_function <- function(.fun, args = list(), setup = NULL, name = NULL
    , n = 10) {
16     results <- vector("list", n)
17
18     for (i in seq_len(n)) {
19         if (!is.null(setup)) setup()
20
21         mem_before <- mem_used()
22         t_start <- Sys.time()
23
24         do.call(.fun, args)
25
26         t_end <- Sys.time()
27         mem_after <- mem_used()
28
29         results[[i]] <- list(
30             step = name,
31             run_id = i,
32             time_ms = as.numeric(difftime(t_end, t_start, units = "secs")) * 1000,
33             memory_MB = as.numeric(mem_after - mem_before) / (1024^2)
34         )
35     }
36
37     return(bind_rows(results))
38 }
39 # Convert("./data/counts.h5ad", dest="h5seurat", overwrite=TRUE)
40 counts <- LoadH5Seurat("./data/counts.h5seurat")
41 pbmc <- CreateSeuratObject(counts = counts[["RNA"]], project = "pbmc3k", min.
    cells = 3, min.features = 100)
42 pbmc_copy <- unserialize(serialize(pbmc, NULL))
43
44 ##### quality control #####
45 filter_result <- benchmark_function(
46     subset,
47     args = list(
48         x = pbmc,
49         subset = expr(nFeature_RNA > 100 & nFeature_RNA < 8000 & nCount_RNA <
            140000)
50     ),

```

```

51  setup = function() {
52    pbmc <- unserialize(serialize(pbmc_copy, NULL))
53  },
54  name = "filter",
55  n = benchmark_times
56 )
57 pbmc = subset(pbmc_copy, subset = nFeature_RNA > 100 & nFeature_RNA < 8000 &
    nCount_RNA < 140000)
58
59
60 pbmc[["percent.mt"]] <- PercentageFeatureSet(pbmc, pattern = "^MT-")
61 pbmc[["percent.ribo"]] <- PercentageFeatureSet(pbmc, pattern = "^RPS|^RPL")
62 pbmc[["percent.hb"]] <- PercentageFeatureSet(pbmc, pattern = "^HB[^P]")
63 pbmc_copy = unserialize(serialize(pbmc, NULL))
64
65 plot1 <- VlnPlot(pbmc, features = c("percent.mt", "nFeature_RNA", "nCount_RNA"),
    pt.size = 0.1, ncol = 3)
66 ggsave("./img/r/qc_violin.png", plot1, width = 18, height = 8)
67
68 plot1 <- FeatureScatter(pbmc, feature1 = "nCount_RNA", feature2 = "percent.mt")
69 plot2 <- FeatureScatter(pbmc, feature1 = "nCount_RNA", feature2 = "nFeature_RNA")
70 ggsave("./img/r/qc_scatter.png", plot1+plot2, width = 12, height = 6)
71
72 ##### normalization #####
73 normalize_result <- benchmark_function(
74   NormalizeData,
75   args = list(
76     pbmc,
77     normalization.method = "LogNormalize"
78   ),
79   setup = function() {
80     pbmc <- unserialize(serialize(pbmc_copy, NULL))
81   },
82   name = "normalize",
83   n = benchmark_times
84 )
85 pbmc <- NormalizeData(pbmc_copy, normalization.method = "LogNormalize")
86 pbmc_copy = unserialize(serialize(pbmc, NULL))
87 # normalized data sorted in `pbmc[["RNA"]]$data`
88
89 ##### highly variable genes #####
90 hvg_result <- benchmark_function(
91   FindVariableFeatures,

```

```

92     args = list(
93         pbmc,
94         selection.method = "vst",
95         nfeatures = 2000
96     ),
97     setup = function() {
98         pbmc <- unserialize(serialize(pbmc_copy, NULL))
99     },
100     name = "hvg",
101     n = benchmark_times
102 )
103 pbmc <- FindVariableFeatures(pbmc_copy, selection.method = "vst", nfeatures =
    2000)
104 # to get hvg gene, use `VariableFeatures(pbmc)`
105 top10 <- head(VariableFeatures(pbmc), 10)
106
107 plot1 <- VariableFeaturePlot(pbmc)
108 plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
109 ggsave("./img/r/hvg.png", plot1+plot2, width = 12, height = 6)
110
111 ##### dimensionality reduction #####
112 pbmc <- ScaleData(pbmc, features = VariableFeatures(pbmc))
113 pbmc_copy = unserialize(serialize(pbmc, NULL))
114 pca_result <- benchmark_function(
115     RunPCA,
116     args = list(
117         pbmc,
118         features = VariableFeatures(pbmc),
119         npcs = 15
120     ),
121     setup = function() {
122         pbmc <- unserialize(serialize(pbmc_copy, NULL))
123     },
124     name = "pca",
125     n = benchmark_times
126 )
127 pbmc <- RunPCA(pbmc_copy, features = VariableFeatures(pbmc), npcs = 15)
128 pbmc_copy = unserialize(serialize(pbmc, NULL))
129
130 plot1 <- ElbowPlot(pbmc, ndims = 15)
131 ggsave("./img/r/variance_ratio.png", plot1, width = 6, height = 4)
132
133 ##### clustering #####
134 neighbors_result <- benchmark_function(
135     FindNeighbors,

```

```
136  args = list(  
137      pbmc,  
138      dims = 1:15  
139  ),  
140  setup = function() {  
141      pbmc <- unserialize(serialize(pbmc_copy, NULL))  
142  },  
143  name = "neighbors",  
144  n = benchmark_times  
145  )  
146  pbmc <- FindNeighbors(pbmc_copy, dims = 1:15)  
147  pbmc_copy = unserialize(serialize(pbmc, NULL))  
148  clustering_result <- benchmark_function(  
149      FindClusters,  
150      args = list(  
151          pbmc,  
152          resolution = 0.5,  
153          graph.name = "RNA_snn"  
154      ),  
155      setup = function() {  
156          pbmc <- unserialize(serialize(pbmc_copy, NULL))  
157      },  
158      name = "clustering",  
159      n = benchmark_times  
160  )  
161  pbmc <- FindClusters(pbmc_copy, resolution = 0.5, graph.name = "RNA_snn")  
162  pbmc_copy = unserialize(serialize(pbmc, NULL))  
163  
164  umap_result <- benchmark_function(  
165      RunUMAP,  
166      args = list(  
167          pbmc,  
168          dims = 1:15,  
169          min.dist = 0.5,  
170          n.neighbors = 50  
171      ),  
172      setup = function() {  
173          pbmc <- unserialize(serialize(pbmc_copy, NULL))  
174      },  
175      name = "umap",  
176      n = benchmark_times  
177  )  
178  pbmc <- RunUMAP(pbmc_copy, dims = 1:15, min.dist = 0.5, n.neighbors = 50)  
179  
180  plot1 <- DimPlot(pbmc, reduction = "umap")
```

```

181 ggsave("./img/r/umap_louvain.png", plot1, width = 6, height = 5)
182
183 ##### benchmark results #####
184
185 all_results <- bind_rows(
186   filter_result,
187   normalize_result,
188   hvg_result,
189   pca_result,
190   neighbors_result,
191   clustering_result,
192   umap_result
193 )
194 write_csv(all_results, "./result/R/benchmark_all.csv")

```

scanpy 速度检测

```

1  #! /home/lihuax/miniconda3/envs/scanpy/bin/python3
2
3  import time
4  import psutil
5  import copy
6  import numpy as np
7  import pandas as pd
8  import scanpy as sc
9  import anndata as ad
10
11
12  def benchmark_function(func, args=None, setup=None, name=None, n=10):
13      if args is None:
14          args = []
15
16      process = psutil.Process()
17
18      results = []
19
20      for i in range(n):
21          if setup is not None:
22              setup()
23              args[0] = adata
24              mem_before = process.memory_info().rss
25              t_start = time.time()
26              func(*args)
27              t_end = time.time()
28              mem_after = process.memory_info().rss

```

```

29         time_ms = (t_end - t_start) * 1000 # 转换为毫秒
30         memory_MB = (mem_after - mem_before) / (1024**2) # 转换为MB
31
32         results.append(
33             {"step": name, "run_id": i + 1, "time_ms": time_ms, "memory_MB":
34                 memory_MB}
35         )
36     return pd.DataFrame(results)
37
38
39 def setup_function():
40     global adata
41     adata = copy.deepcopy(adata_backup)
42
43
44 bench_times = 100
45
46
47 adata = ad.read_h5ad("./data/counts.h5ad")
48 adata_backup = copy.deepcopy(adata)
49
50
51 ##### quality control #####
52 def filter_cells_min_genes_func(adata, min_genes=100):
53     sc.pp.filter_cells(adata, min_genes=min_genes)
54
55
56 def filter_genes_min_cells_func(adata, min_cells=3):
57     sc.pp.filter_genes(adata, min_cells=min_cells)
58
59
60 def filter_cells_max_genes_func(adata, max_genes=2000):
61     sc.pp.filter_cells(adata, max_genes=max_genes)
62
63
64 def filter_cells_max_counts_func(adata, max_counts=10000):
65     sc.pp.filter_cells(adata, max_counts=max_counts)
66
67
68 filter_cells_min_genes_result = benchmark_function(
69     filter_cells_min_genes_func,
70     args=[adata, 100],
71     setup=setup_function,
72     name="filter_cells_min_genes",

```

```

73     n=bench_times,
74 )
75 filter_genes_min_cells_result = benchmark_function(
76     filter_genes_min_cells_func,
77     args=[adata, 3],
78     setup=setup_function,
79     name="filter_genes_min_cells",
80     n=bench_times,
81 )
82 filter_cells_max_genes_result = benchmark_function(
83     filter_cells_max_genes_func,
84     args=[adata, 2000],
85     setup=setup_function,
86     name="filter_cells_max_genes",
87     n=bench_times,
88 )
89 filter_cells_max_counts_result = benchmark_function(
90     filter_cells_max_counts_func,
91     args=[adata, 10000],
92     setup=setup_function,
93     name="filter_cells_max_counts",
94     n=bench_times,
95 )
96
97 adata = copy.deepcopy(adata_backup)
98 sc.pp.filter_cells(adata, min_genes=100)
99 sc.pp.filter_genes(adata, min_cells=3)
100 sc.pp.filter_cells(adata, max_genes=2000)
101 sc.pp.filter_cells(adata, max_counts=10000)
102
103 adata.var["mt"] = adata.var_names.str.startswith("MT-")
104 adata.var["ribo"] = adata.var_names.str.startswith(("RPS", "RPL"))
105 adata.var["hb"] = adata.var_names.str.contains("^HB[^(P)]")
106 adata_backup = copy.deepcopy(adata)
107
108
109 def calculate_qc_metrics_func(adata):
110     sc.pp.calculate_qc_metrics(
111         adata, qc_vars=["mt", "ribo", "hb"], inplace=True, log1p=True
112     )
113
114
115 calculate_qc_metrics_result = benchmark_function(
116     calculate_qc_metrics_func,
117     args=[adata],

```



```

118     setup=setup_function,
119     name="calculate_qc_metrics",
120     n=bench_times,
121 )
122
123 adata = copy.deepcopy(adata_backup)
124 sc.pp.calculate_qc_metrics(
125     adata, qc_vars=["mt", "ribo", "hb"], inplace=True, log1p=True
126 )
127 adata = adata[adata.obs.pct_counts_mt < 11, :]
128 adata = adata[adata.obs.pct_counts_mt > 0.5, :]
129 sc.pl.violin(
130     adata,
131     ["n_genes_by_counts", "total_counts", "pct_counts_mt"],
132     jitter=0.4,
133     multi_panel=True,
134     save=True,
135 )
136 sc.pl.scatter(
137     adata,
138     "total_counts",
139     "n_genes_by_counts",
140     color="pct_counts_mt",
141     save=True,
142 )
143
144 ##### normalization #####
145 adata.layers["counts"] = adata.X.copy()
146 adata_backup = copy.deepcopy(adata)
147 normalize_total_result = benchmark_function(
148     sc.pp.normalize_total,
149     args=[adata],
150     setup=setup_function,
151     name="normalize_total",
152     n=bench_times,
153 )
154 adata = copy.deepcopy(adata_backup)
155 sc.pp.normalize_total(adata)
156 adata_backup = copy.deepcopy(adata)
157
158 log1p_result = benchmark_function(
159     sc.pp.log1p,
160     args=[adata],
161     setup=setup_function,
162     name="log1p",

```

```

163     n=bench_times,
164 )
165 adata = copy.deepcopy(adata_backup)
166 sc.pp.log1p(adata)
167 adata_backup = copy.deepcopy(adata)
168
169
170 ##### highly variable genes #####
171 def highly_variable_genes_func(adata, n_top_genes=2000):
172     sc.pp.highly_variable_genes(adata, n_top_genes=n_top_genes)
173
174
175 hvg_result = benchmark_function(
176     highly_variable_genes_func,
177     args=[adata, 2000],
178     setup=setup_function,
179     name="highly_variable_genes",
180     n=bench_times,
181 )
182 adata = copy.deepcopy(adata_backup)
183 sc.pp.highly_variable_genes(adata, n_top_genes=2000)
184 sc.pl.highly_variable_genes(adata, save=True)
185 adata_backup = copy.deepcopy(adata)
186
187 ##### dimensionality reduction #####
188 pca_result = benchmark_function(
189     sc.tl.pca,
190     args=[adata],
191     setup=setup_function,
192     name="pca",
193     n=bench_times,
194 )
195 adata = copy.deepcopy(adata_backup)
196 sc.tl.pca(adata)
197 sc.pl.pca_variance_ratio(adata, n_pcs=50, log=True, save=True)
198 adata_backup = copy.deepcopy(adata)
199
200 ##### clustering #####
201 neighbor_result = benchmark_function(
202     sc.pp.neighbors,
203     args=[adata],
204     setup=setup_function,
205     name="neighbors",
206     n=bench_times,
207 )

```

```

208 adata = copy.deepcopy(adata_backup)
209 sc.pp.neighbors(adata)
210 adata_backup = copy.deepcopy(adata)
211
212 leiden_result = benchmark_function(
213     sc.tl.leiden,
214     args=[adata],
215     setup=setup_function,
216     name="leiden",
217     n=bench_times,
218 )
219 adata = copy.deepcopy(adata_backup)
220 sc.tl.leiden(adata, flavor="igraph", n_iterations=2)
221 adata_backup = copy.deepcopy(adata)
222
223 umap_result = benchmark_function(
224     sc.tl.umap,
225     args=[adata],
226     setup=setup_function,
227     name="umap",
228     n=bench_times,
229 )
230 adata = copy.deepcopy(adata_backup)
231 sc.tl.umap(adata)
232 sc.pl.umap(adata, color=["leiden"], save=True)
233
234 ##### benchmark results #####
235
236 all_results = pd.concat(
237     [
238         filter_cells_min_genes_result,
239         filter_genes_min_cells_result,
240         filter_cells_max_genes_result,
241         filter_cells_max_counts_result,
242         calculate_qc_metrics_result,
243         normalize_total_result,
244         log1p_result,
245         hvg_result,
246         pca_result,
247         neighbor_result,
248         leiden_result,
249         umap_result,
250     ],
251     ignore_index=True,
252 )

```

```
253  
254 all_results.to_csv("./result/python/benchmark_results.csv", index=False)
```

致 谢

本论文的完成离不开多方的关心与支持，在此谨致以诚挚的感谢。

首先，衷心感谢我的导师赵伟老师在课题选题、研究设计与论文撰写过程中给予的悉心指导与宝贵建议。赵老师严谨治学的态度、敏锐的问题意识以及持续探索的精神，为本研究的顺利推进提供了重要保障。

感谢生命科学学院为我提供了良好的学习与科研环境，系统的课程设置与丰富的学术资源，为我的专业成长和科研训练打下了坚实基础。感谢各位授课教师在本科阶段的辛勤教学和耐心指导，使我得以不断夯实专业知识，拓展学术视野。

感谢我的家人长期以来给予的理解与支持，是他们无声的鼓励与陪伴，使我能够专注于学业，克服困难，坚定前行。

同时，感谢开源社区与相关软件项目开发者的无私贡献，特别是 **Muon.jl**、**Scanpy** 等项目，为本研究提供了重要的参考与技术支持。

最后，感谢在项目开发、测试与讨论过程中给予我帮助与建议的同学与朋友们。你们的支持使本项目不断完善，持续前进。

毕业论文（设计）成绩表

导师评语

导师评价

建议成绩 0

指导教师（签字） 赵伟

答辩委员会意见

答辩委员会负责人（签字） _____

成绩 _____

学院（盖章） _____

年 月 日